

Using Unix commands to analyze text documents & introduction to UNICODE

Christophe Servan, christophe_DOT_servan_AT_universite-paris-saclay_DOT_fr

Inspired from "Unix for Poets" de Kenneth Ward Church, IBM Research, [kwchurch \(AT\) us.ibm.com](mailto:kwchurch@us.ibm.com)

1 Program

1. Count words in text
2. Order a list of words by alphabetical
3. Calculate occurrence and co-occurrence statistics
4. Extract useful information from a dictionary

Tools

- `cat/head/tail` : display (the whole / the beginning / the end) of a file on the screen
- `less` : display a file with scrolling
- `grep` : search for a pattern (regular expression)
- `sort`
 - : sort the lines
- `uniq -c` : eliminate or count duplicates
- `tr` : transpose or eliminate characters
- `wc` : count lines, words and characters
- `sed` : edit (replace, delete) a string with regular expressions
- `awk` : language for examining and processing patterns
- `cut` : extract columns
- `paste` : "paste" columns
- `join` : equivalent to SQL JOIN
- `iconv` : Conversion of character encodings :
- Web browser : Mozilla Firefox, Google Chrome or Chromium
- Hexedit text editor
- Python3

1.1 Suggestions

always remember to read the tool's `man` page if you have any doubts about the tool's parameters and functionality. For example, the `man wc` command indicates how you can count only lines (`-l`) or only words (`-w`) in a file.

2 Input / output redirection

By default, most command line tools read the input (`stdin`) from the keyboard and write the output (`stdout`) to the screen. You can redirect inputs and outputs to files using the following commands :

<	Read from an input file
>	Write to an output file
	Write the output of the previous command to the input of the next command (pipe)

For example, sort the 100th first lines of a file and put it into another one :

```
$ head -n 100 file | sort > myoutputfile
```

3 Data

File for exercises : [RADIOS.txt](#). This file contains transcripts of radio recordings : Franceinter, RFI,...

4 Work to do

For each exercise, you will need to compare the command line and the equivalent in python (don't use jupyter notes, use python scripts).

4.1 Exercise 1

Transform the text into `RADIOS.txt` into capital letters and put it into another file

4.2 Exercise 2

Find the sequence of instructions that allows you to count words in `RADIOS.txt`. Put the list of words with their number of appearances in an output file (`RADIOS.hist`)

Help : use `tr`, `sort` and `uniq`, think of "piping" (!) the instructions

4.2.1 Questions :

- **Q1** : What word appears exactly 1,732 times in this text ?
- **Q2** : How many times does the word "orange" appear in this text ?

4.3 Exercise 4 - Sort the words (sort)

See man page of `sort`

Examples :

<code>sort -d</code>	dictionary order
<code>sort -f</code>	ignore upper / lower case
<code>sort -n</code>	numerical order
<code>sort -nr</code>	reverse numerical order
<code>sort -k 1</code>	start at field 1 (the first is field 0)
<code>sort -k 0.50</code>	start at the 50 th character
<code>sort -k 1.5</code>	start at 5 th character of the field 1

4.3.1 Questions

- **Q1** : Sort the words of `RADIO.txt` by frequency of appearance
- **Q2** : Sort the words of `RADIO.txt` in alphabetical order
- **Q3** : Sort the words of `RADIO.txt` in "rhymic" order (example, put together all the words which end with "-ment").

Help : use the command `unix rev`

4.4 Exercise 3 : Co-occurrences of words or bigrams

We will find and count all the bigrams text `RADIOS.txt`

- Input : `RADIOS.txt` text file
- Output : statistics file where each line is of the form "word1 word2 nb"

4.4.1 Questions : Do this for n-grams (n = 1,2,3 and 4).

- **Q1** : How many times does the 2-gram "it is" appear in this text ?
- **Q2** : How many separate 3-grams are found in this text ?
- **Q3** : What is the most common 4-gram ?

Help : use the `tail` and `paste` commands

4.5 Exercise 5 : filters (grep)

See man page of grep.

Examples

grep '[A-Z]'	lines containing a capital letter
grep '^ [A-Z]'	lines starting with a capital letter
grep '[A-Z]\$'	lines ending with a capital letter
grep '^ [A-Z]*\$'	fully uppercase lines
grep "[aeiouAEIOU]"	lines containing a vowel
grep "^ [aeiouAEIOU]"	lines starting with a vowel
grep "[aeiouAEIOU]\$"	lines ending with a vowel
grep -i '[aeiou]'	lines containing a vowel
grep -i '^ [aeiou]'	lines starting with a vowel
grep -i '[aeiou]\$'	lines ending with a vowel
grep -i '^[^aeiou]'	lines starting with a non-vowel
grep -i '[^aeiou]\$'	lines ending with a non-vowel
grep -i '[aeiou].*[aeiou]'	lines with at least two vowels
grep -i '^[^aeiou]*[aeiou][^aeiou]*\$'	lines with exactly one vowel

With regular expressions

a	the letter "a"
[a-z]	a tiny letter
[A-Z]	a capital letter
[0-9]	a number<
[0123456789]	a number
[aeiouAEIOU]	a vowel
[^aeiouAEIOU]	anything but a vowel
.	a character
^	start of line
\$	end of line
x*	"x" repeated 0 or more times
x+	"x" repeated 1 or more times
x y	"x" or "y" (only egrep)
(x)	impose rule previously (only egrep)

4.5.1 Questions

- Q1 : How many has the words of 9 letters in RADIOS.txt ?
- Q2 : Are Has -it words without vowels in RADIOS.txt

4.6 Exercise 6 : Awk language

awk is a language whose syntax is similar to C, and which allows operations to be performed on fields in a file where each line is of the type "field1 field2 field3 field4..."

Example

- `awk '{print $1}'` selects the first field in a file, equivalent to `cut -f 1`
- `awk '$1 > 100 { print $0}' RADIOS.hist` displays the words whose number of occurrences is greater than 100 in RADIOS.txt

It is possible to write the program in a file and after calling it, for example, by typing `SelectPremierChamp.awk`

The available predicates are `:`, `<`, `>`, `=`, `<=`, `==`, `!=`, `&&`, `||`

- **Q1** : Find the bi-grams that appear exactly 13 times.
- **Q2** : Find the palindromes in RADIOS.txt (words written in the same way from right to left or from left to right).

4.7 Exercise 7 : Replacement with sed

See man page of sed.

The sed tool allows you to replace text using regular expressions. Thus, the command `sed 's/exp1/exp2/[op]` replace (s) the expression `exp1` by the expression `exp2`. The option is often the letter `g` to say that all occurrences on the same line should be replaced.

For example :

- `sed 's/[éèêë]/e/g'` replaces all accented letters "e" with an unaccented letter
- `sed 's/ \([^]*\)ation / \U\1ATION/g'` transforms the words which end with the suffix "ation" into capital letters, for example, "habitation" becomes "HABITATION"

4.7.1 Questions

- **Q1** : Add an end point to each line and put the first letter in capital letters.
- **Q2** : Replace any occurrence of two consecutive identical words with a single occurrence of that word, for example, "de de" becomes "de".

The End